

Yenepoya Institute of
Arts, Science, Commerce
and Management
Project – High Level Design
on
Bigdata Analytics on Smart City Demand Analysis

Fathimath Shakiya A S

Industry Mentor
Sumit Kumar Shukla

Guided by
Prashantha J

Table of Contents

1. Introduction
2. Scope of the Document
3. Intended Audience
4. System Overview
5. System Design
6. Application Design
7. Process Flow
8. Information Flow
9. Components Design
10. Key Design Considerations
11. API Catalogue
12. Data Design
13. Data Model
14. Data Access Mechanism
15. Data Retention Policies
16. Data Migration
17. Interfaces
18. State and Session Management
19. Caching
20. Non-Functional Requirements
21. Security Aspects
22. Performance Aspects
23. References

1. Introduction

This document provides a high-level design for the project **“Big Data Analytics for Smart City Demand.”**

The system analyzes urban demand patterns such as traffic, electricity, and water usage using Python, Pandas, and SQL.

It simulates big data processing using manageable datasets to demonstrate real-world analytics workflows.

2. Scope of the Document

This document describes:

- Overall system architecture
- Data processing pipeline (ETL)
- Analytical workflows
- Storage and reporting mechanisms

3. Intended Audience

- Students (BCA/BSc)
- Faculty evaluators
- Project reviewers
- Beginners in Data Science
-

4. System Overview

The system processes smart city data to identify demand trends using:

- **Input:** Historical smart city dataset
- **Processing:** Data cleaning, transformation, analysis
- **Output:** Insights, graphs, reports

5. System Design

Architecture Type:

- Standalone analytical system

Technologies Used:

- Python
- Pandas
- SQLite
- Jupyter Notebook
- Seaborn / Matplotlib

6. Application Design

The application is divided into:

- Data ingestion module
- Data transformation module
- Data analysis module
- Visualization module
- Storage module

7. Process Flow

1. Load dataset (CSV)
2. Clean missing values
3. Perform feature engineering
4. Apply GroupBy aggregations
5. Generate visualizations
6. Store processed data in SQLite
7. Run SQL queries for reporting

8. Information Flow

CSV Dataset → Pandas Processing → Analysis → Visualization → SQLite Database → Reports

9. Components Design

Key Components:

1. **Data Loader** : Reads CSV data
2. **Data Processor** : Cleans and transforms data
3. **Analytics Engine** : Performs aggregations and correlations
4. **Visualization Engine** : Generates charts using Seaborn
5. **Database Module** : Stores results in SQLite

10. Key Design Considerations

- Simplicity for student-level implementation
- Efficient data handling using Pandas
- Clear visualization for insights
- Modular design for scalability

11. API Catalogue

No external APIs are used.

All processing is done locally using dataset files.

12. Data Design

Data Includes:

- Traffic volume
- Weather conditions
- Time features (hour, day)
- Demand levels (electricity, water, transport)

13. Data Model

Structured tabular format:

Column	Description
<i>hour</i>	Time of day
<i>day_of_week</i>	Day index
<i>traffic_volume</i>	Traffic count
<i>weather_main</i>	Weather type
<i>temp</i>	Temperature
<i>demand_levels</i>	Categorized demand

14. Data Access Mechanism

- Data accessed using **Pandas DataFrames**
- SQL queries executed via **SQLite**

15. Data Retention Policies

- Data stored locally in SQLite database
- No long-term retention required
- Data used only for academic purposes

16. Data Migration

- CSV → Pandas → SQLite

17. Interfaces

- Jupyter Notebook interface
- Command-based execution
- Visualization output via plots

18. State and Session Management

- No session persistence required
- Each notebook run processes data independently

19. Caching

- No caching implemented
- Data processed in-memory using Pandas

20. Non-Functional Requirements

- Usability: Easy to understand for students
- Reliability: Accurate data processing
- Maintainability: Modular code structure

21. Security Aspects

- No sensitive data involved
- Local execution only
- No external data exposure

22. Performance Aspects

- Efficient processing using Pandas
- Handles medium-sized datasets
- Fast query execution using SQLite

23. References

- Python Documentation
- Pandas Documentation
- SQLite Documentation
- Open-source smart city datasets